SQL Foundations - Understanding the Language

WELCOME TO SQL
Today's SQL Foundations - Before You Start Coding
Expectation - Understand what SQL is and why it matters

The Question Everyone Has: "What actually is SQL?"

The Confusion

You've heard terms like CREATE, SELECT, INSERT, UPDATE, DELETE, SHMT... But these all the same thing? Are some more important than others? Which one do I need to learn first? Do I need to memorize syntax?

The Truth

- SQL isn't one thing
- It's actually FIVE different types of commands
- Each type has a different purpose

Think of it like Engine:

- Gas pedal controls acceleration
- Brakes control speed
- Steering wheel controls direction

Different parts, different purposes.

Today's Goal

- Understand the five command types
- Know which one you're using when
- Recognize what you're looking at in code

What You'll Learn In 15 Minutes

- The 5 types of SQL commands (DDL, DML, DQL, DCL, TCL)
- Which one you're using in any given situation

Why This Matters

Understanding the structure of EVERYTHING that comes next

Why This Matters

Without this foundation, Video 1 might confuse you

With this foundation, Video 1 will make sense immediately

THE LIBRARY ANALOGY

Understanding SQL Through a Library

Imagine You're Managing a Library

Building the Library (DDL)
- Creating new sections
- Expanding buildings
- Organizing sections (Fiction, Non-Fiction, Reference)
- Changing layouts
- Getting up the zoning system

This is **DDL = Data Definition Language**
You're defining the STRUCTURE

Managing the Books (DML)
- Adding new books to inventory
- Updating book information (new edition, change location)
- Removing lost/damaged books
- Re-organizing books within sections

This is **DML = Data Manipulation Language**
You're managing the actual CONTENTS
The actual everyday work - you're filling it

Finding Books (DQL)
- Searching the catalog
- Checking what's available
- Where is the book X filed under?
- "How many books do we have in Science Fiction?"
- Reading library cataloging availability

This is **DQL = Data Query Language**
You're READING information

And nothing anything, not changing anything
Just looking at what's there

Library Access Cards (DCL)
- Who can access the building?
- Who can add new books?
- Who can update the catalog?
- Student card vs Staff card vs Admin card

This is **DCL = Data Control Language**
You're controlling WHO can do WHAT
Security and permissions

Protecting Rare Books (TCL)
- Checking out a book (temporarily borrowing)
- If you drop a rare book, return it to original condition
- If 7 need to reorganize, but first make sure the catalog still works

This is **TCL = Transaction Control Language**
You're creating safety nets
The "undo" button for databases

The Complete Picture

All five types work together to run a library

All five types work together to run a database

THE FIVE TYPES EXPLAINED

DDL, DML, DQL, DCL, TCL - What They Actually Mean

TYPE 1: DDL - Data Definition Language

Purpose: Define the STRUCTURE of your database tables

Main Commands:
- CREATE - Build new tables/databases
- ALTER - Modify existing structures
- DROP - Delete structure permanently
- TRUNCATE - Remove all data but keep the structure

Library Analogy:
- CREATE TABLE employees = A Dulo employees table
- ALTER TABLE employees ADD middle_phone = Add new column
- DROP TABLE old_backup = Delete old backup table

Library Analogy:
- Installing new bookshelves
- Changing shelf labels
- Removing old card catalog

When You Use It:
- Setting up a new database
- Adding columns to tables
- Modifying existing structures

Key Thing:
Video 17 (Dim Warehouse projects)
You'll CREATE TABLE to build analytics systems

TYPE 2: DML - Data Manipulation Language

Purpose: Manipulate the ACTUAL DATA inside tables

Main Commands:
- INSERT - Add new rows
- UPDATE - Change existing rows
- DELETE - Remove rows

Real Examples:
- INSERT INTO employees VALUES (= Add new employee
- UPDATE employees SET salary = SENIOR WHERE id = 5 = Give raise
- DELETE FROM employees WHERE status = 'terminated' = Remove record

Library Analogy:
- Adding new books to shelf (INSERT)
- Update book location or edits info (UPDATE)
- Remove damaged books (DELETE)

When You Use It:
- Loading data into databases
- Correcting errors in data
- Removing outdated records

⚠ WARNING: UPDATE and DELETE are PERMANENT
Always test with SELECT first
Use transactions (TCL) for safety

In The Course:
Video 17 (Dim Warehouse projects)
You'll INSERT INTO to build analytics systems

TYPE 3: DQL - Data Query Language

Purpose: Search and question around the data

Main Command:
- SELECT - Retrieve/read data (completely safe, read-only)

Real Examples:
- SELECT * FROM employees = Show me all employees
- SELECT COUNT(*) FROM employees WHERE salary > 70 = How many
- SELECT dept, AVG(salary) FROM employees GROUP BY dept = Summary

Library Analogy:
- Searching the catalog
- Checking what's available
- Reading cataloging information
- Exploring data

When You Use It:
- Everyday work
- Building reports
- Building reports and dashboards
- Exploring data

The Beauty of SELECT:
- Completely safe - never changes anything
- You can run SELECT queries all day, safe as you want
- If you make a mistake, just run it again differently

⚠ THIS IS SQL, NOT YOUR JOB AS ANALYST

In The Course:
Video 1: 15 Hours hands-on SQL
Video 1 teaches SELECT to all 5 Analysts
WHERE, GROUP BY, HAVING, SUBQUERIES - all DQL

TYPE 4: DCL - Data Control Language

Purpose: Control WHO can access WHAT

Main Commands:
- GRANT - Give permissions to users/roles
- REVOKE - Remove permissions

Real Examples:
- GRANT SELECT ON employees TO analyst_role = Read access only
- GRANT SELECT, UPDATE ON sales TO manager_role = Read and update
- REVOKE DELETE ON employees FROM analyst_role = Remove delete rights

Library Analogy:
- Student card vs Staff card vs Admin card
- Current scanner (read-only, DRAWING)
- Student cards can only borrow (SELECT)
- Librarian cards can add books (INSERT)

When You Use It:
- Database administrators (DBAs)
- Security teams
- IT managers

Not a Junior Analyst:
- You probably WON'T write GRANT/REVOKE
- Your company's DBA sets permissions
- You work within the permissions you're given

In The Course:
Mentioned for completeness
Understand the concept, don't memorize commands

TYPE 5: TCL - Transaction Control Language

Purpose: Control TRANSACTIONS (groups of changes)

Main Commands:
- BEGIN TRANSACTION - Start a group of changes
- COMMIT - Save changes permanently
- ROLLBACK - Undo all changes since BEGIN
- SAVEPOINT - Create checkpoint to rollback to

Real Examples:
```
BEGIN TRANSACTION;
UPDATE employees SET salary = salary * 1.10;
-- Oops, forgot the WHERE clause!
ROLLBACK; -- Undo! Put it back to original
```
Library Analogy:
- Begin moving books around
- If you drop a rare book, return it TO/AMY Beep 10
- If you don't like it, ROLLBACK (put books back)

The Safety Net:
Imagine you're UPDATING WRONG
You accidentally change 10,000 rows
WITHOUT transaction: Permanent disaster!
WITH transaction: Just ROLLBACK, back to state
⚠ CRITICAL: NEVER - Always BEGIN TRANSACTION before UPDATE/DELETE
Test your changes
ROLLBACK if wrong, COMMIT if right

In The Course:
Best practices mentioned throughout
Used in advanced exercises
Essential for production work

WHAT YOU'LL USE IN THIS COURSE

Which SQL Types Matter for Data Analysts?

Video 1-4: Foundation (90% DQL)

What you'll write:
```
SELECT * FROM sales WHERE sale > 100;
SELECT customer, SUM(sale) FROM sales GROUP BY product;
SELECT customer, COUNT(*) FROM sales GROUP BY customer HAVING COUNT(*) > 5;
```
Why DQL dominates:
- Analysts spend more time READING data, not changing it
- Business questions = SELECT queries
- Reports = SELECT queries
- Reports = SELECT queries

Skills you'll master:
- Filtering (WHERE)
- Aggregating (GROUP BY)
- Joining tables (INNER/LEFT JOIN)
- Aggregating (SUM, COUNT, AVG, MIN/MAX)

Videos 5-16: Intermediate/Advanced (Still mostly DQL)

What you'll add:
- Subqueries
- CTEs (Common Table Expressions)
- Window functions (RANK, ROWNUM, NVMR)

Still all DQL!

You're asking more sophisticated questions

But still reading data, not changing it

Videos 17-20: Data Warehouse (DDL + DML + DQL)

Now you'll use ALL types:

DDL:
```
CREATE TABLE fact_sales (...);
ALTER TABLE dim_customers ADD ...;
CREATE INDEX idx_customer_id ON customer(customer_id);
```
DML:
```
INSERT INTO fact_sales SELECT * FROM staging_sales;
UPDATE dim_customers SET status = CHRTRIGGERED;
```
DQL:
```
SELECT * FROM gold_sales_summary WHERE rank <= 20;
```
Why this matters:
- Data engineers BUILD databases (DDL)
- Data engineers LOAD data (DML)
- Data analysts QUERY databases (DQL)
- A warehouse project, you play both roles

Throughout: TCL (Best Practices)

Every time you UPDATE or DELETE:

```
BEGIN TRANSACTION;
UPDATE employees SET salary = salary * 1.10;
-- Check: SELECT COUNT(*) FROM ... WHERE ... = 10,000
-- Looks right?
COMMIT;
-- Wrong? ROLLBACK;
```
This habit saves careers

One wrong UPDATE - not correct production data

Transactions give you an undo button

COMMON MISTAKES & HOW TO AVOID THEM

Learn These Patterns Now, Save Hours Later

Mistake 1: UPDATE/DELETE Without WHERE

The Disaster:
```
UPDATE employees SET salary = 50000;
-- Oops, everyone's salary is now 50k
```
Oops just ate EVERY employee's salary

The Fix:
```
-- Always test with SELECT first:
SELECT * FROM employees
WHERE department = 'Sales';
-- Looks right? Now add UPDATE:
UPDATE employees SET salary = 50000
WHERE department = 'Sales';
```
Use BEGIN TRANSACTION before every complex change

Gives you ROLLBACK option if you make a mistake

Mistake 2: Not Testing Your Changes First

The Pattern:
- Write complex UPDATE
- Run it directly on production
- Hope it worked
- Discover it didn't

```
BEGIN TRANSACTION;
UPDATE customers SET status = 'VIP'
WHERE total_spent > 10000;
-- Check: SELECT COUNT(*) FROM customers WHERE status = 'VIP';
-- Looks right?
COMMIT;
-- Wrong? ROLLBACK;
```
The Fix:
- Always test SELECT before UPDATE/DELETE
- Use BEGIN TRANSACTION for safety
- Check results before COMMIT
- Keep ROLLBACK as escape hatch

The Professional Way:
```
-- Step 1: Preview
SELECT * FROM customers
WHERE total_spent > 10000;
-- Step 2: Count affected rows
SELECT COUNT(*) FROM customers
WHERE total_spent > 10000;
-- Step 3: Make the change
BEGIN TRANSACTION;
UPDATE customers SET status = 'VIP'
WHERE total_spent > 10000;
-- Step 4: Verify
SELECT COUNT(*) FROM customers
WHERE status = 'VIP';
-- Step 5: Decide
COMMIT;
-- OR
ROLLBACK;
```
Mistake 3: Confusing DDL and DML

The Confusion:
"I need to add a phone number for customer 123"

Thinking: ALTER TABLE customers ADD phone_123 VARCHAR(20);

The Reality:
- DDL changes STRUCTURE (adding new column)
- DML changes data (updating one row)

Right (DML):
```
UPDATE customers SET phone
= '555-1234' WHERE id = 123;
```
Wrong (DDL):
```
ALTER TABLE customers
ADD phone_123 VARCHAR(20);
```
This updates SAME for an existing customer

Why:
- DDL = Change structure (add/remove/rename columns, tables)
- DML = Change data (add/remove/update rows)

YOU'RE READY FOR VIDEO 1

Foundation Complete - Let's Solve Business Problems!

What You Now Understand

The 5 SQL Types:
- DDL - Define structure (CREATE, ALTER, DROP)
- DML - Manipulate data (INSERT, UPDATE, DELETE)
- DQL - Query data (SELECT)
- DCL - Control access (GRANT, REVOKE)
- TCL - Control transactions (BEGIN, COMMIT, ROLLBACK)

What Matters for Analysts:
- 90% of your work = DQL (SELECT queries)
- Video 1-4 = Pure DQL (reading and analyzing data)
- Review = DCL, results DMLs do DML
- Advanced work = DCL (security team's job)
- Always = TCL (safety habit for UPDATE/DELETE)

You Can Now Recognize:
- "This is a DDL query because it's SELECT"
- "This is DML because it's INSERT/UPDATE/DELETE"
- "This is DCL because it's GRANT/REVOKE"

The Safety Mindset:
- SELECT is completely safe
- UPDATE/DELETE need transactions
- Always test before running on real data
- WHERE clause is your friend

Video 1: 15 Hours Ahead

Video 1 teaches SELECT to all 5 analysts
You now understand WHY we're using SELECT
You know what SELECT is doing in the grand scheme
You now have the foundation to learn everything else

In Video 1 (Today):

You'll use SELECT DISTINCT to answer: A giant business questions
Or your first day as a data analyst at Acme!
Sales, Marketing, Finance, Ops, Product, Sales, AND, JOIN
CTEs, Subqueries, Window functions

Ready? Let's go!

Video 1: The Task: Analyst's First Day

Mission Time: 20 minutes

Business Question: "Which stores in Wisconsin?" or "Which products in Q4?"

Your Tools: SELECT, WHERE, GROUP BY, HAVING, JOIN

Your Goal: Answer business values on Day 1

Remember: You've learned the foundation. Now apply it to real business problems.